SIGMA CHI WASHU ·
EXPERIMENT/BEER-DIE-REDESIGN

Beer Die Home Page

How the home page becomes a die table on scroll: what we would build, what we would build it with, and the fourteen mockups that explore it, including the revised Glow Toss, its cup-sink ending and two holographic throwers.



Prepared November 5, 2026
For the colony's website, sigma-chi-website repository
Branch experiment/beer-die-redesign · mockups in beer-die-redesign/mockups

1 · SUMMARY

No new framework. A new layer.

The effect fits the site's current stack. Vite, React 19 and Motion already handle scroll on the live hero; the missing piece is a 3D renderer for the table and die. After the first round of ten mockups, your feedback asked for something more realistic, more modern and glowier, with far less scrolling. Mockup 11 does that, mockup 12 is the same build ending in a cup, 13 and 14 add a holographic player who throws it (14 with a heavyweight build), and this plan builds toward them.

Keep

Vite 8, React 19, TypeScript, react-router 7 and Motion 13. Motion detects the first scroll and runs the intro clock.

Add

`three` through React Three Fiber 9, drei 10 and `@react-three/postprocessing` for bloom, lazy-loaded as one chunk on the home page only.

Skip

GSAP, runtime physics engines, Spline and a framework migration. Each one duplicates Motion, needs a CSP change, or adds weight without changing the result.

The brief, as mockup 11 builds it

- **It loads looking normal.** The first frame is today's hero: same headline, standfirst, buttons, pinstripes and crest watermark. The table is there, seen straight down and lit flat, so it reads as the familiar navy field.
- **One scroll starts it.** The first bit of scroll, or a click on the cue, plays the whole sequence on its own clock in about 3.4 seconds. Nobody has to scroll four screens to see it. Scrolling back to the top rewinds it.
- **The background turns out to be the table.** The camera cranes up from overhead to a player's view while gold light strips trace the inlay and center line, and four glass cups rise at the corners.
- **The crest becomes the die and gets thrown.** The watermark flies onto the die's top face. The toss uses real gravity: about 5 ft of air, two bounces that lose energy, and a damped settle that always ends crest-up. "Toss again" throws it back from the other end, as players alternate in a real game.
- **Same palette.** Every color comes from `src/styles/theme.css`; gold is the only color that glows. Type stays Cormorant Garamond and Source Sans 3, plus JetBrains Mono for the small HUD readouts.

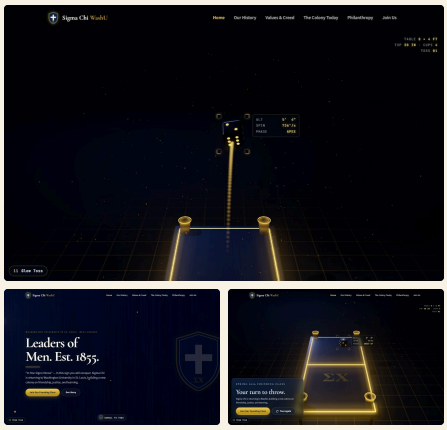
Navy #0B2545	Navy dark #071931	Gold #C9A227	Gold glow #F0CF62 (new)	Gold dark #8A6D1B	Cream #FAF7F0

Before this ships: get sign-off on the theme. Beer die is a drinking game, and the home page is the colony's recruitment page while it builds standing with WashU's IFC and the national fraternity. Every mockup keeps the imagery dry (empty cups, no beer, no cans), and none of the on-page copy says "beer". Run the concept past the colony's risk manager or expansion coordinator before merging to `main`.

2 · FOURTEEN MOCKUPS

Ten ways to throw it, then four more

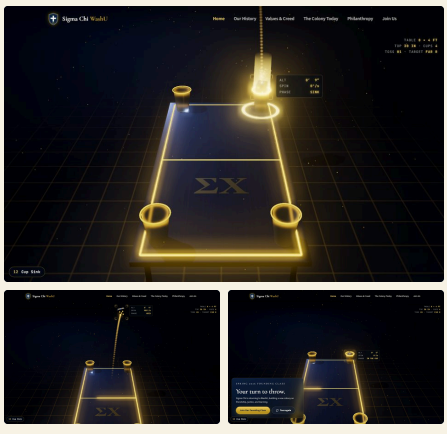
Each mockup is one self-contained page in `beer-die-redesign/mockups/`, also published as a private artifact. The first ten each test a different reveal and, where useful, a different technology. Number 11 is the revision after your feedback, 12 is 11 with the die going into a cup, 13 adds a holographic player who throws it, and 14 gives him a heavyweight build. The large frame is each concept's signature moment; all of them open on today's hero, except 10, which never leaves it.



11 Glow Toss **REVISION · RECOMMENDED**

The direction after your feedback. It has physically based materials, neon gold inlays, glass cups, bloom, and a HUD that reads out the die's height and spin. One small scroll plays the whole toss in about 3.4 seconds with real ballistics, and "Toss again" throws it back the other way.

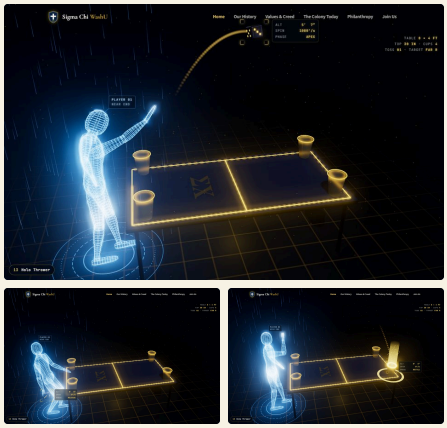
Built with three.js 0.160, bloom post-processing and a custom studio environment map. Triggered by scroll but runs on its own clock; pinned for under one screen.



12 Cup Sink **VARIANT OF 11**

11 with a different ending. The die clears the rim on a gravity arc, drops in, rattles against the wall and settles crest-up, and the cup answers with a gold flash, a short column of light and sparks. "Toss again" alternates ends and works round all four cups.

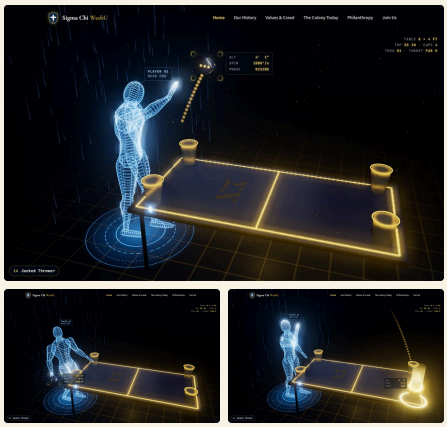
Built with the same stack as 11. The cups are drawn 8 in across and the die a little smaller so it fits inside.



13 Holo Thrower **12 WITH A PLAYER**

A featureless figure drawn only in blue contour lines prints itself in from the floor at the near end, through falling blue code. It lobs the die underhand, watches it, and pumps a fist when it sinks. The camera ends side-on, so the swing reads in profile and the arc crosses the frame.

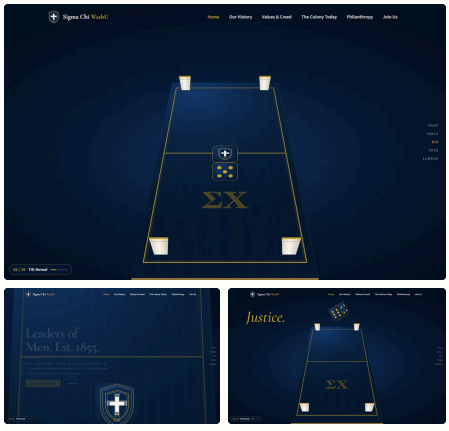
Built with the same stack as 12. The figure is generated in code, with no model file: lofted rings, a joint hierarchy and leg IK.



14 Jacked Thrower **13, HEAVYWEIGHT**

13 with a heavyweight build: broad shoulders and lats over a narrow waist, pecs, a six-pack, big delts and arms, quads and diamond calves. Brighter lines trace the muscle borders. The camera ends on a front three-quarter view so the build faces you, and the arc still crosses to the cups.

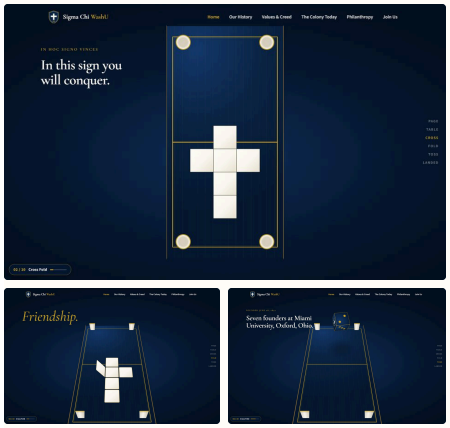
Built with the same code as 13. Each cross-section is an ellipse swollen by Gaussian muscles, and the proportions are a table of numbers.



01 Tilt Reveal

The hero's navy field tilts back and turns out to be the painted table. The crest watermark flies onto the die's top face, and the die arcs over the line and lands crest-up.

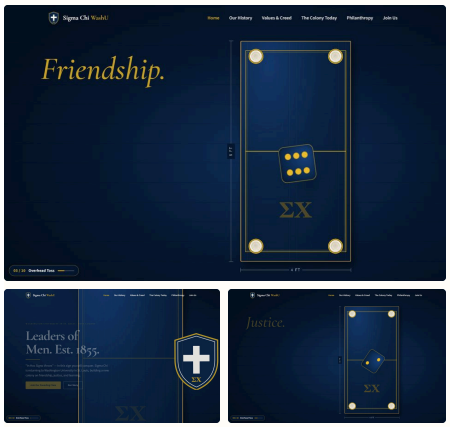
Built with CSS 3D transforms and vanilla JS, scrubbed by scroll over 3.6 screens.



02 Cross Fold

The crest's White Cross drops onto the table as six squares, because an unfolded die is a cross. The squares fold up on hinges into the die, which is then thrown.

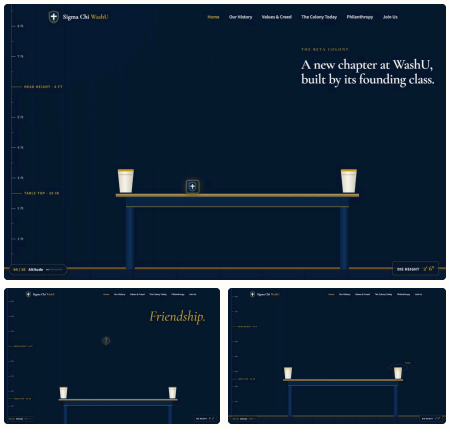
Built with CSS 3D hinges and vanilla JS, scrubbed over 4.2 screens.



03 Overhead Toss

The camera stays straight overhead while the page zooms out to a 4 × 8 ft table. The toss reads through the die's growing scale and a shadow that slides away and softens.

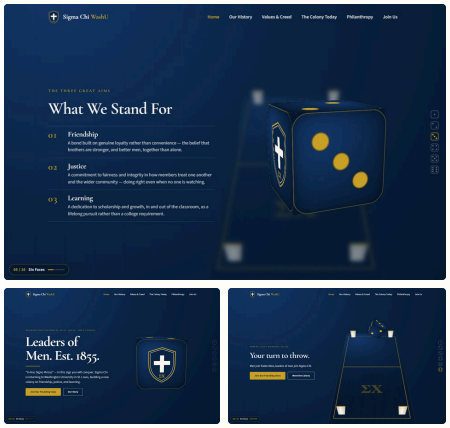
Built with one SVG and 2D transforms, no 3D at all, scrubbed over 3.4 screens. The cheapest and most robust option.



04 Altitude

A side view drawn to scale. The view climbs with the die past a ruler (table top 30 in, head height 6 ft, apex 10 ft), and the die drops into the far cup.

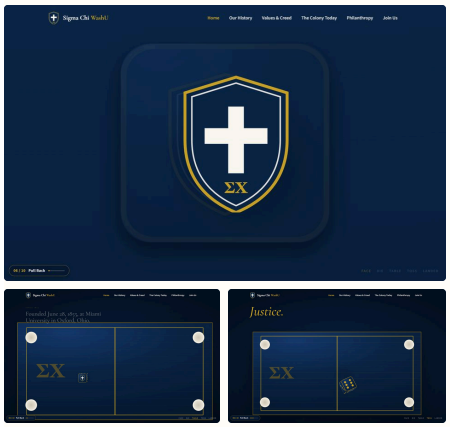
Built with GSAP 3.12 and ScrollTrigger, pinned and scrubbed over 4.5 screens.



05 Six Faces

The crest becomes face 1 of a large pinned die. Each chapter of the page turns it to its next face while the table rises behind it out of focus, and the last chapter throws it onto the table.

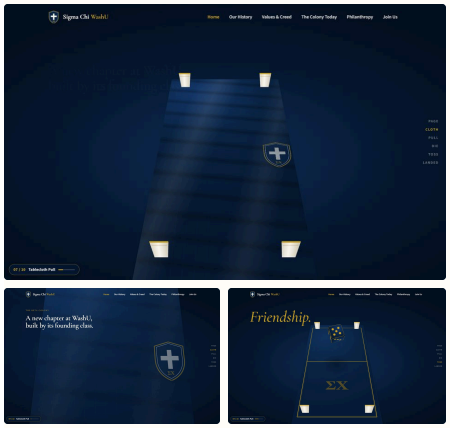
Built with Motion's `scroll()` and `animate()` and a CSS 3D die, scrubbed over 7 screens.



06 Pull Back

The hero shrinks into a rounded square that turns out to be the die's top face. The camera pulls back to the die on the table, then to the whole table, and then the die is thrown.

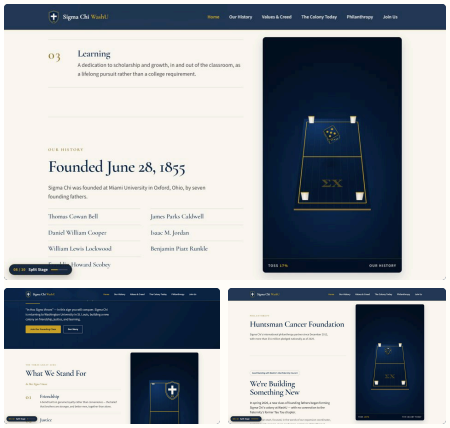
Built with a log-space 2D zoom and a CSS 3D die, scrubbed over 3.8 screens.



07 Tablecloth Pull

The hero is a navy cloth over the table. Scrolling yanks it off sideways while the cups wobble in place, and the crest printed on it stays behind as the die.

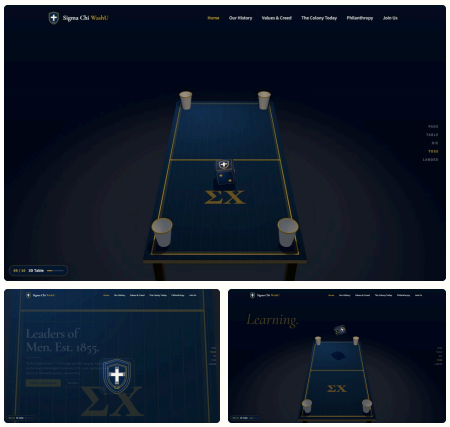
Built with CSS 3D, skew and an SVG displacement filter, scrubbed over 4 screens.



08 Split Stage

The page reads normally on the left while a sticky panel on the right holds the table, and the toss follows reading progress. On phones the panel becomes a band under the nav.

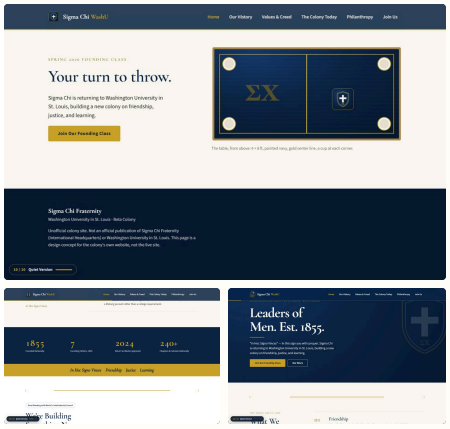
Built with Motion `scroll()` with two targets and a CSS 3D table. No extra scroll; it rides along with the page.



09 3D Table

The first WebGL build: a lit, rounded die, a painted table on legs, lathe-turned cups and soft shadows. The camera cranes from straight down to a player's view.

Built with three.js 0.160 through an importmap, scrubbed over 3.6 screens.



10 Quiet Version

Today's home page, nearly untouched. The nav crest becomes a small die that tumbles with reading progress, and a small table at the end catches it beside the Join button.

Built with native CSS scroll-driven animations with a small JS fallback. No libraries and no extra scroll.

3 · SIDE BY SIDE

How the concepts compare

Scroll is how far the reader has to scroll while the scene is pinned. Effort is for a production build in the site's React code, not the mockup.

#	CONCEPT	FITS THE BRIEF	SCROLL	PRODUCTION TECH	EFFORT	PHONE RISK
11	**Glow Toss**	**Strong** and realistic	**One flick**; pinned 0.75 screen	R3F + drei + bloom	Large	Medium (bloom, glass)
12	**Cup Sink**	**Strong**, ending in a cup	**One flick**; pinned 0.75 screen	R3F + drei + bloom	Large	Medium (bloom, glass)
13	**Holo Thrower**	**Strong**, with a player	**One flick**; pinned 0.75 screen	R3F + drei + bloom, plus a rig	Large+	Medium (more draw calls)
14	**Jacked Thrower**	**Strong**, with a heavyweight player	**One flick**; pinned 0.75 screen	R3F + drei + bloom, plus a rig	Large+	Medium (39 draw calls for the figure)
01	Tilt Reveal	**Strong**	3.6 screens	R3F, or CSS 3D	Medium	Low
02	Cross Fold	**Strong** and the most symbolic	4.2 screens	R3F hinged planes	Medium–large	Medium
03	Overhead Toss	**Partial**: the toss reads as a zoom	3.4 screens	SVG + Motion	Small	Low
04	Altitude	**Partial**: side-on, but the best "up"	4.5 screens	SVG + Motion	Medium	Low
05	Six Faces	**Partial**: die first, table second	7 screens	CSS 3D + Motion	Medium	Medium (blur filter)
06	Pull Back	**Strong**: best "it looked normal" twist	3.8 screens	CSS 3D + Motion	Medium	Low
07	Tablecloth Pull	**Strong**: the most theatrical	4 screens	CSS 3D + SVG filter	Large	High (displacement filter)
08	Split Stage	**Partial**: the most readable	None extra	CSS 3D + Motion	Small–medium	Low
09	3D Table	**Strong**	3.6 screens	R3F + drei	Large	Medium
10	Quiet Version	**Light**: the theme as an accent	None extra	CSS only	Small	Low

Recommendation: build 11 or 12. They're one build with two endings: 11 lands crest-up on the table and 12 sinks the die in a cup. Picking one is a matter of taste, and the code could even ship both, with the ending chosen per toss. Either one is the brief made the way you asked for in the second round: realistic, glowing, and started by one flick of the wheel instead of four screens of scrolling. 13 and 14 show that a thrower can be layered onto either ending with no change to the stack; 14's build is the same code with different numbers. Section 8 lists what every effect needs. Two first-round ideas are worth keeping in reserve. 02's cross fold is an alternative crest-to-die moment that ties the die to the White Cross. 10's nav die could carry the theme onto the other pages without a second 3D scene.

Why 11 plays on its own clock

Mockups 01 to 09 scrub: every pixel of scroll moves the scene a little, so the toss lasts as long as the reader keeps scrolling (three to seven screens). That's what made them feel like a lot of mouse work. 11 is scroll-triggered instead. The first downward scroll starts a fixed-length timeline, and the reader can keep scrolling, or not, while it plays. It's also why the toss can look physical: gravity needs real seconds, not scroll pixels. The cost is that the reader no longer controls the speed, so the sequence is kept under five seconds (the WCAG 2.2.2 threshold for moving content), it can be skipped, and reduced motion jumps straight to the landed frame.

4 · FRAMEWORKS AND LIBRARIES

What we would build it with

You asked whether this needs a different framework. It doesn't. Scroll handling and timing are already Motion's job on this site; the only missing capability is drawing a lit 3D die and table.

OPTION	WHAT IT GIVES US	COST	VERDICT
Vite + React 19 + Motion 13 already installed	Scroll position (`useScroll`), a time-based clock (`animate` on a motion value) that can run forward or in reverse, and transforms on the HTML layers.	None	**Keep** The base for every concept.
three.js via React Three Fiber 9 + drei 10	Physically based materials, a rounded die, glass cups, soft shadows, a studio environment built from drei `Lightformer` s (no downloaded HDR), and a camera rig, all inside the React tree.	Roughly 250–300 KB gzipped with post-processing (estimate), as a lazy chunk	**Use** R3F 9 is the release line that supports React 19.
@react-three/postprocessing	Bloom that picks up only the gold: inlay strips, cup rims, pips and the flight trail.	Included above	**Use** This is where the glow comes from.

OPTION	WHAT IT GIVES US	COST	VERDICT
CSS 3D transforms	A crisp cube and table with no dependencies. Mockups 01, 02, 05, 06, 07 and 08 use them.	None	**Not for 11** There's no lighting, bloom or glass, and Safari flattens 3D under `opacity`, `filter` or `overflow`.
GSAP + ScrollTrigger	Excellent timelines and pinning, free for all uses since 2025. Mockup 04 uses it.	About 35 KB gzipped	**Skip** It duplicates Motion, which is already in the bundle.
Physics engines Rapier, cannon-es	Simulated tumbles.	Hundreds of KB of WASM and a CSP change (`'wasm-unsafe-eval'`)	**Skip** 11's toss is closed-form physics: a gravity parabola, energy-losing bounces and a damped settle. That's a few dozen lines, it's deterministic, and it always lands crest-up.
Spline / Rive runtimes	Designer-authored scenes.	Large runtime; hosted assets are blocked by the site CSP	**Skip** Less control and vendor lock-in.
Native CSS scroll-driven animations	Scroll-linked animation with no JavaScript. Mockup 10 uses it.	None	**Small touches** Fine for a nav die on other pages, with the JS fallback.
Switching frameworks Next.js, Astro	Server rendering, islands.	A migration	**No** The effect is entirely client-side, and the site deploys as static files to GitHub Pages.
Smooth-scroll and scroll-lock libraries	Inertial or hijacked scrolling.	Small	**No** 11 never takes over the scrollbar. The page keeps scrolling normally while the toss plays.

Dependencies the build would add

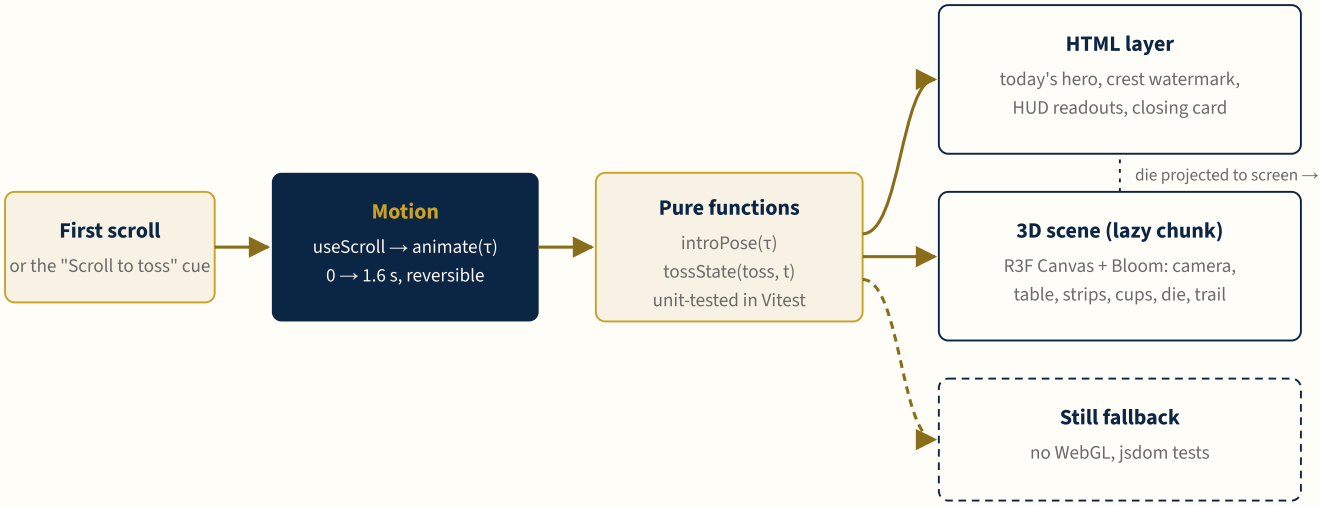
```
# runtime, loaded in a lazy chunk on the home page only
npm i three @react-three/fiber@^9 @react-three/drei@^10 @react-three/postprocessing
# types and visual regression tests
npm i -D @types/three @playwright/test
```

Nothing else changes in `package.json`. Motion stays at 13.x. Pin exact versions in the lockfile as usual. The HUD face, JetBrains Mono, gets self-hosted through `@fontsource` like the site's other fonts, because the CSP only allows `font-src 'self'`.

5 · ARCHITECTURE

One clock, two layers

The stage is a short section, 175vh, with a sticky viewport. Motion watches the scroll position. The first downward scroll animates an intro clock from 0 to 1.6 seconds, and returning to the top animates it back to 0 at double speed. Pure functions turn that clock, plus the current toss's own clock, into every pose. The HTML layer (headline, crest watermark, HUD, closing card) and the 3D scene read the same values, so they can't drift apart.



Files

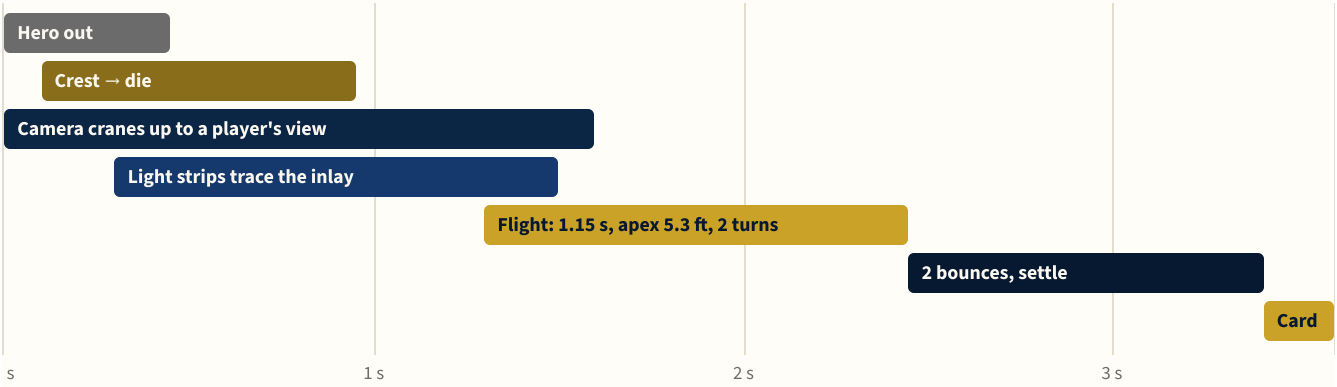
PATH	JOB
`src/features/dieTable/DieTableHero.tsx`	Replaces `HeroBanner` on the home page only. Renders the stage, today's hero markup, the cue, HUD, closing card and skip link, and picks between the 3D scene and the still.
`src/features/dieTable/useIntroClock.ts`	Starts the intro on the first scroll and rewinds it at the top. Returns a `MotionValue<number>` in seconds.
`src/features/dieTable/introPose.ts`	Camera crane, hero fade, crest handoff, light-strip trace and cup rise, as a function of the intro clock.
`src/features/dieTable/tossPhysics.ts`	`makeToss(from, to, seed)` and `tossState(toss, t)`: flight, bounces, settle, spin, and phase labels for the HUD.
`src/features/dieTable/*.test.ts`	Lands crest-up, never sinks into the table, apex height matches gravity, the first frame equals today's hero, and rewinding returns to it.

PATH	JOB
`src/features/dieTable/scene/DieTableScene.tsx`	The lazy R3F `Canvas`: studio environment, lamp, table, strips, cups, die, trail, impact ring and `Bloom`.
`src/features/dieTable/scene/faceTextures.ts`	Draws the `ShieldCrest` paths and the pips into colour, glow and bump canvases for the die, so the crest has one source of truth.
`src/features/dieTable/StillFrame.tsx`	A static landed composition for no WebGL and for tests.
`src/components/ShieldCrest.tsx`, `src/pages/Home.tsx`	Export the crest path constants, and swap `HeroBanner` for `DieTableHero` on the home page. Other pages are untouched.

6 · CHOREOGRAPHY

The toss, second by second

Timings from mockup 11. The intro runs 1.6 seconds; the toss is released at 1.3 seconds and has its own clock, so "Toss again" can replay it without the intro. The scene is built in feet, so gravity is 32.2 ft/s².



Bounce heights are 0.5 ft and 0.1 ft, so they last 2√(2h/g): 0.35 s and 0.16 s. The settle is a 0.45 s damped wobble onto a whole number of turns, which is what guarantees crest-up. Impacts fire a gold ring on the table and a tiny camera shake. Mockup 12 shares the intro; its flight (1.1 s) ends just over a cup's rim, followed by a 0.16 s drop inside, a small hop off the bottom and a 0.55 s rattle that settles crest-up.

The core of it in code

```
// useIntroClock.ts: one scroll plays it, the top of the page rewinds it
export function useIntroClock() {
  const tau = useMotionValue(0);
  const lastTarget = useRef(0);
  const reduce = useReducedMotion();
  const { scrollY } = useScroll();
  useMotionValueEvent(scrollY, "change", (y) => {
    const target = y > 6 ? INTRO_END : 0;
    if (target === lastTarget.current) return;
    lastTarget.current = target;
    const distance = Math.abs(target - tau.get());
```

```
    animate(tau, target, { ease: "linear", duration: reduce ? 0 : target ? distance : distance / 2 });
  });
  return tau;
}

// tossPhysics.ts: closed-form, deterministic, always lands crest-up
export function tossState(k: Toss, t: number): TossState {
  if (t < k.flight) {
    const u = t / k.flight;                       // apex = G * flight² / 8 ≈ 5.3 ft
    return { ...lerp2(k.from, k.p1, u), height: 4 * k.apex * u * (1 - u), angle: k.A1 * u, phase:
phaseOf(u) };
  }
  // ...two shorter parabolas for the bounces, then a damped settle onto A4 = 3 full turns
}

// scene/Rig.tsx: lift the die so its lowest corner touches the table at any orientation
const e = m4.makeRotationFromQuaternion(q).elements;
die.position.y = (HALF - ROUND) * (Math.abs(e[1]) + Math.abs(e[5]) + Math.abs(e[9])) + ROUND +
state.height;
```

7 · THE DIE AND THE TABLE

Realistic materials, one glowing color

DIE

- `RoundedBox` with 6 segments and a 15% edge radius.
- `MeshPhysicalMaterial`: navy lacquer, clearcoat 1. Three canvases per face: colour (gold pips, the crest on face 1), glow (emissive, so the pips bloom) and bump (so the pips read as engraved).
- Opposite faces sum to seven. It's drawn larger than a real 5/8 in die so it stays legible at table scale.

TABLE

- 4 × 8 ft, 30 in off the floor, with a brushed gold-dark metal trim and slim dark legs.
- The top is navy lacquer with fine grain and the hero's 96 px pinstripes. **Its gloss is tied to the camera:** matte while the camera looks straight down, so the hero reads as a flat field, then clear-coated once the camera is looking along the table. Without that, the lamp's reflection blooms into a white blob behind the headline.
- The inlay and center line are thin HDR gold strips that trace on during the crane. The ΣX monogram is its own layer and fades in.

CUPS

- Frosted glass (`transmission`), empty, with a lit gold rim. On weak devices, swap the glass for an opaque cream material; transmission costs an extra render pass.

LIGHT AND REFLECTIONS

- The environment map is built in code from drei `Lightformer`s: a navy void, one low softbox behind the far end, a cool side panel and a gold strip. Reflections stay navy and gold; a stock "room" environment turns the lacquer grey.
- One neutral, slightly cool key spot, placed off-axis, with soft shadows. The warmth comes only from the gold.
- Bloom threshold 0.9, so only the gold glows. Its strength ramps from 0.25 at the hero to 0.75 once the table is revealed.

THE THROWER (MOCKUPS 13 AND 14)

- Built in code: each body part is a loft of rings (the blue contour lines) over a faint rim-lit shell. In 14, the rings

- The floor has a faint gold grid that fades out from under the table.

swell into muscles. There's no model file, so the CSP stays as it is.

- A plain joint hierarchy is posed from keyframes (idle, wind-up, release, follow-through, fist pump). Two-bone IK keeps the feet planted as the hips drop and rise.
- The die rides in the hand until release. The launch velocity is then solved from the release point, so gravity carries it exactly into the cup.
- Section 8 breaks down every effect, including two ways to build the figure for production.

MOTION DETAILS

- The flight trail re-samples the same deterministic path slightly in the past, so it needs no stored state.
- A gentle pointer parallax after the intro, and no camera roll ever.

What each effect needs

Every visual effect in mockups 11 to 14: how it's made, and what it takes to build for production. None of them needs a library beyond those in section 4. Each one is a custom shader, some generated geometry, or a few dozen lines of math.

The table, the die and the toss (11–14)

EFFECT	HOW IT'S MADE	WHAT IT NEEDS	COST
Glow	Gold and blue parts render brighter than white, and a bloom pass blurs only what's above a 0.9 threshold. Tone mapping runs last.	`Bloom` from `@react-three/postprocessing` with `mipmapBlur`; a half-float render target	The biggest GPU cost. Run it at half resolution on phones.
Lacquered table	A canvas-painted navy top with grain and the hero's pinstripes, under a clear-coated physical material. Gloss rises with the camera crane, so the hero view stays matte.	`MeshPhysicalMaterial`; one 2048 × 1024 canvas	Low
Light-strip inlay	Thin boxes in HDR gold, scaled along their length on the intro clock so the border and center line trace themselves on.	10 small meshes (can be merged)	Low
Reflections	A studio environment built in code: navy void, one softbox, a cool side panel, a gold strip. No HDR download.	drei `Environment` with `Lightformer`s	One-off at load
Engraved, glowing pips	Three canvases per face: colour, glow (so the pips bloom) and bump (so they read as engraved). The crest is drawn from the `ShieldCrest` paths.	drei `RoundedBox`; 18 small canvases	Low
Glass cups	Lathe-turned cups in frosted glass, with an HDR gold rim that flashes on a sink and stays lit while the die is inside.	`MeshPhysicalMaterial` with `transmission`	Medium: transmission adds a render pass. Use opaque cream on weak devices.
Toss physics	Closed-form: a gravity parabola, energy-losing bounces or a cup drop, then a damped settle onto whole turns, so it always lands crest-up. When the figure throws (13–14), the launch is solved from the release point: $v_0 = (P_1 - P_0)/T + \tfrac{1}{2}gT$, straight up.	A pure TypeScript module, unit-tested	None at runtime
Flight trail	The same path sampled a little in the past, drawn as additive glow points that fade with speed.	`Points`; one sprite	Low

EFFECT	HOW IT'S MADE	WHAT IT NEEDS	COST
Impact and sink	An expanding gold ring on the table and a short camera shake. In a cup: a column of light, gold sparks and a cup wobble.	A ring, an open cylinder, a 36-point burst	Low
HUD	HTML over the canvas. The die's eight corners are projected to the screen each frame for the reticle; the readouts come from the physics module.	React and CSS; `Vector3.project`	Low
Crest hand-off	The HTML crest moves onto the projected rectangle of the die's top face, then the 3D die takes over in the same spot.	Motion values and projection	Low
Atmosphere	Drifting dust, a gold floor grid that fades out, film grain and a vignette.	`Points`; a canvas; CSS overlays	Low. Dust is the only thing that moves at rest.

The holographic thrower (13–14)

EFFECT	HOW IT'S MADE	WHAT IT NEEDS	COST
Contour-line body	Each body part is a loft: rings of a changing cross-section along the part, plus a few long seams, drawn as 1-pixel lines in an additive shader.	Generated `BufferGeometry` and `LineSegments`; a short GLSL shader	About 5,900 line segments in 14
The jacked build (14)	Each cross-section is an ellipse swollen by smooth Gaussian bulges placed by angle and height: pecs, a six-pack, lats, traps, delts, biceps, triceps, quads and calves. Brighter lines trace the muscle borders (ab grid, pec line, delt V, calf split) so the build reads from the front.	A table of numbers per body part; no assets	Tuning time, not runtime
Rim-lit volume	A matching closed surface with a fresnel shader. It glows only at its silhouette, so the lines read as a solid body.	The same generator; a 10-line shader	About 8,400 triangles of additive overdraw
Print-in and scan	The shader hides everything above a rising height and brightens the print edge. A soft band travels up the body, with a faint flicker.	Three uniforms driven by the intro clock	Negligible
The throw	A joint hierarchy posed from keyframes (idle, wind-up, release, follow-through, fist pump) with eased transitions. Two-bone IK keeps the feet planted as the hips drop and rise, and the head tracks the die.	About 120 lines of TypeScript	Negligible at runtime; 1–2 days to feel right
Code rain	About 110 short falling streaks around the thrower, brightest at the leading end, surging while he prints in.	`LineSegments`; move the fall into a vertex shader for production	Negligible
Pad and blue spill	A canvas-drawn projector pad under his feet, and a blue point light at his chest so he lights the table.	One plane; one extra light	Low, but the light adds work to every lit material

EFFECT	HOW IT'S MADE	WHAT IT NEEDS	COST
Reload	On "Toss again", the die shrinks out of the cup in blue sparks and grows back in his hand.	Reuses the spark burst	Negligible

Two ways to build the thrower

A · GENERATED IN CODE (WHAT 13 AND 14 DO)

- No assets, no licences, no 3D software. Every proportion is a number, which is how 14 got jacked: same code, different table.
- The CSP stays exactly as it is.
- About 39 draw calls for the figure (lines, borders and shell for each part), because each part moves on its own. That's fine on a laptop. On phones, drop the shell first.
- The motion is hand-keyed, so it looks stylised rather than motion-captured.

B · A RIGGED BODY WITH A CONTOUR SHADER

- A muscular base mesh, sculpted or from a CC0 or licensed library, rigged to a standard skeleton. Throw and celebration clips come from motion capture (Mixamo has several; confirm its licence for a public site).
- No line geometry: a shader draws the contours from world height (`fract(y × density)`) plus the fresnel rim, so any mesh gets the same look in two draw calls.
- Load it with drei's `useGLTF` and `useAnimations`. An untextured GLB served from the site keeps the CSP unchanged; embedded textures would need `blob:` in `img-src`.
- It needs Blender for retargeting and clean-up: 2–4 extra days, and someone comfortable in it.

> **Recommendation: ship A first.** It already exists, it's easy to tune, and it carries no asset or licence risk. Move to B only if you want lifelike motion or a library of animations, such as different celebrations or idle loops.

Line weight

WebGL draws lines 1 device pixel wide everywhere, so on a 3× phone screen the contours are hairlines. That suits a hologram. For bolder lines, three's `LineSegments2` and `LineMaterial` draw true screen-space widths at roughly three times the vertex cost; route B's shader controls the width directly.

What it has to get right

ACCESSIBILITY

- All words stay HTML. The canvas is `aria-hidden`; the HUD is decorative.
- The sequence runs about 3.4 seconds, under WCAG 2.2.2's five-second limit for moving content. It's skippable ("Skip past the toss"), and the cue is a real button for keyboard users.
- **Reduced motion jumps straight to the landed frame.** No crane, no trail, no shake, no drifting dust. The app's `MotionConfig reducedMotion="user"` doesn't cover a WebGL scene, so `DieTableHero` checks `useReducedMotion()` itself.
- The page never locks or hijacks the scrollbar.
- The thrower follows reduced motion too: he appears fully printed, with no flicker, scan band or code rain, and the page shows the landed state instead of the throw.

SECURITY (THE SITE'S CSP)

- The current policy works as-is: three, R3F and post-processing ship in our bundle (`script-src 'self'`); textures come from canvases; the environment map is generated in code, with no HDR download.
- Self-host JetBrains Mono through `@fontsource` (`font-src 'self'`).
- The policy would need changing only for runtime physics (`'wasm-unsafe-eval'`), GLB textures (`blob:`) or a CDN-hosted Draco decoder. The plan avoids all three.

PERFORMANCE BUDGET

First paint and LCP	Unchanged. The LCP element stays the hero headline, and the page's own gradient shows until the canvas fades in.
3D chunk	≤ 300 KB gzipped, fetched on idle after load.
Frame rate	60 fps on an iPhone 12-class phone; check with Chrome's 4× CPU throttle.
Rendering	Only while the stage is on screen. Full rate while the intro or a toss runs; dust motes are the only idle motion, and they stop on low-power devices.
Scaling down	drei `PerformanceMonitor` steps down: device pixel ratio 1.75 → 1.25, then glass cups → opaque, then bloom at half resolution, then shadows off.
Thrower (13–14)	At most 6,000 line segments, 10,000 shell triangles and 40 draw calls. On the weakest phones, drop the shell and the code rain first.

FALLBACKS

Reduced motion	The scroll trigger jumps to the landed frame.
No WebGL	Today's hero, then the still and the closing card.
3D chunk still loading	The hero behaves normally; the scene fades in when ready.
Tests (jsdom)	`canRender3D()` returns false, so the still renders.

Testing, rollout and open questions

TESTING

- **Vitest:** `tossPhysics` and `introPose` unit tests (crest-up landing, no table penetration at any orientation, apex = $gT^2/8$, first frame equals today's hero, rewind returns to it).
- **Existing suite:** `App.test.tsx` keeps passing because jsdom takes the still path.
- **Playwright visual checks:** seek the clock to fixed times (hero, apex, impact, landed) on desktop and phone. Mockup 11 already has this hook (`window.__seek`), which is how its frames in this document were captured.
- **By hand:** iOS Safari (sticky with `100svh`), Firefox, reduced motion on, keyboard only, a low-end Android.

PREVIEWING THE BRANCH

GitHub Pages deploys only `main`. Review this branch locally with `npm run dev`, or connect Vercel or Netlify (their configs are already in the repo) for preview URLs. **Don't run the Pages workflow from this branch:** it has `workflow_dispatch`, and a manual run would replace the live site.

ROLLOUT

PHASE	WORK	EFFORT
1	Confirm 11 or 12 as the direction and pick the ending; get theme sign-off.	You
2	Stage, intro clock, `introPose` and `tossPhysics` with tests, HTML layers, still, reduced-motion path.	1–2 days
3	R3F scene: table, strips, cups, die textures, environment, bloom, crest handoff, HUD.	3–5 days
3b	If you want the thrower: figure generator and muscle table, pose keys and leg IK, contour and rim shaders, print-in, code rain, pad.	3–5 days
4	Phone tuning and the performance step-down, Safari pass, Playwright baselines.	2 days
5	Preview deploy, review, merge to `main`.	½ day

These are focused working days with Claude Code doing most of the typing; review and sign-off time is on top.

QUESTIONS FOR YOU

1. Is 11/12 the direction, and do you want the thrower in it, slim (13) or jacked (14)? Or should a first-round idea (02's cross fold, 06's pull back) be folded in?
2. How much HUD: the full readout as mocked up, or just the reticle?
3. Which ending: crest-up on the table (11) or in a cup (12)?
4. Should the glow carry into the rest of the site (the dark glass sections in 11), or stop at the hero?
5. Who signs off on the theme before it goes live?